SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. *(BL3)*

Assessment Tool Weightage: 40%

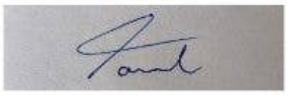
Duration :60 Mins
On Class
QUESTIONS: 5 **Total Marks:50**

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I Tamil Elakiya s (192524443) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

21/7/26

Assessment Tool - 1

Tamil Elakiya S
(192524443)

① **Statement :**

$w = x * y - z + 20$

i) **Lexical Analysis :**

$\langle id_1 \rangle = \langle id_2 \rangle \langle * \rangle \langle id_3 \rangle \langle - \rangle \langle id_4 \rangle \langle + \rangle \langle 20 \rangle$

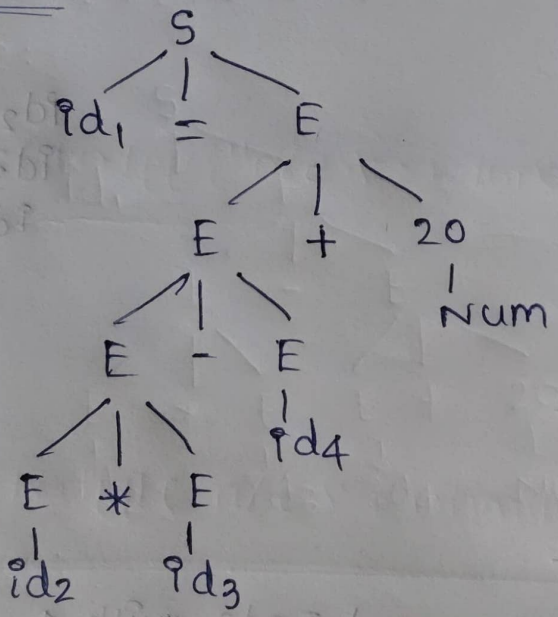
w	id_1	Real	Global	1000	entry-1
x	id_2	Real	Global	1008	entry-2
y	id_3	Real	Global	1016	entry-3
z	id_4	Real	Global	1024	entry-4

ii) **Syntax Analysis :**

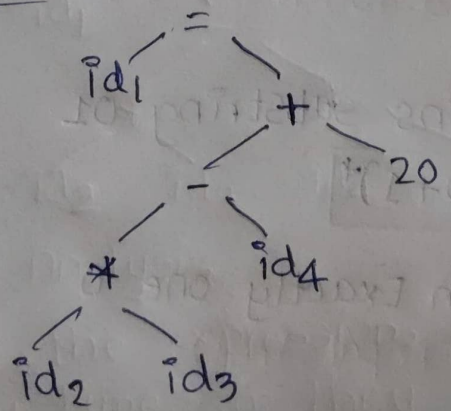
$S \rightarrow id = E$

$E \rightarrow E*E \mid E-E \mid E+E \mid id \mid Num$

Parse tree :



Syntax tree :



iii) Semantic Analysis:

All the Expressions are of same data type; so they are semantically verified and correct.

iv) Intermediate code:

$T_1 = id_2 * id_3$

$T_2 = T_1 - id_4$

$T_3 = T_2 + 20$

$id_1 = T_3$

v) Code optimizer:

$T_1 = id_2 * id_3$

$T_2 = T_1 - id_4$

$id_1 = T_2 + 20$

vi) Code Generator:

```
MOV id2, R3
Mov id3, R2
MUL R3 , R2
Mov id4, R1
SUB R1, R2
ADD #20, R2
MOV R2, id1
```

$id_2 - R_3$
$id_3 - R_2$
$id_4 - R_1$

---

② i) All strings begins and Ends with 1.

$\therefore R = 1(0+1)^*1$

ii) All strings contains substring 01

$\therefore R = (0+1)^*01(0+1)^*$

iii) All strings with Exactly one 0

$\therefore R = 1^*01^*$

iv) All strings with atleast two 1's

$$R = (0+1)^* 1 (0+1)^* 1 (0+1)^*$$

v) All strings with that does not end with '0'.

$$R = (0+1)^* 1 + \varepsilon$$

---

3) Languages Generated for each Regular Expression.

(i) $(\varepsilon + aa^*)^*$

$L = \{\varepsilon, a, aa, aaa, aaaa, \ldots\}$

All strings containing only a's

ii) $(a^* + b^*)^* abb$.

$L = \{abb, aabb, bbabb, ababbb, \ldots\}$

Containg 'abb' as initial value.
and followed by combination of a and b.

iii) $(ab)^* aba$

$L = \{aba, aaba, bbaba, abababa, \ldots\}$.

iv) $(a + ab)^*$

$L = \{\varepsilon, a, ab, aab, aa, aaab, abab, \ldots\}$

v) $(aba)^* + (ab)^*$

$L = \{\varepsilon, ab, aba, abaab, aaa, bbb, \ldots\}$.

---

4) i) Regular Expression:

$$[a\text{-}zA\text{-}z][a\text{-}zA\text{-}z0\text{-}9]^*$$

ii) Justification:

* Given that the identifier must start with a letter.
* Followed by letter there can be combination of letter and digit.

* Identifier must not start with digit.
* Keywords are placed before the identifier, it is considered as keywords.

Code:

```
%{
#include <stdio.h>
%}
%%
if |
else|
while |  {printf("Keyword: %s\n", yytext);}
[a-zA-Z][a-zA-Z0-9]* {printf("Identifier : %s\n", yytext);}
[0-9]+  {printf("constant: %s\n", yytext);}
"+" |
"-" |
"*" |
"/" |  {printf("operator: %s\n", yytext);}
"(" |
")" |
"{" |
"}" |
";"  {printf("Delimiter : %s\n", yytext);}

[ \t\n]+   ;
.   {printf("Invalid Token : %s\n", yytext);}
%%
int yywrap(){
    return 1;
}
int main(){
    printf("Enter the input:\n");
    yylex();
    return 0;
}
```

5) (i) $(0^*1^*)^* = (0+1)^*$

$(0^*1^*)^* = \{\varepsilon, 0, 1, 01, 0011, 1010, \ldots\}$

$(0+1)^* = \{\varepsilon, 0, 1, 01, 0011, 1010, \ldots\}$

For both languages we get same language.

So, $(0^*1^*)^* = (0+1)^*$

(ii) $(0+1)^*(0+1)^* = (0+1)^*$

WKT, $\boxed{r^*.r^* = r^*}$

So, Both generate same language.

$\therefore (0+1)^*(0+1)^* = (0+1)^*$

(iii) $(0^*)^* = 0^*$

Twice the Kleen closure gives the same language as one time Kleen closure.

$\therefore (0^*)^* = 0^*$

(iv) $(0+1)^*0(0+1)^* + (0+1)^*1(0+1)^* = (0+1)^*$

$= (0+1)^*(0+1)^*(0+1)$ ∵ Taking commoning

$= (0+1)^*(0+1)$ ∵ $r^*r^* = r^*$

$= (0+1)^+$ ∵ $r^*r = r^+$

Hence proved.

v) $\varepsilon + 0(0)^* = 0^*$

$= \varepsilon + (0^*)0$ ∵ $r(r)^* = r^+$

$= \varepsilon + r^+$ $\varepsilon + r^+ = r^*$

$= 0^*$

Hence proved.